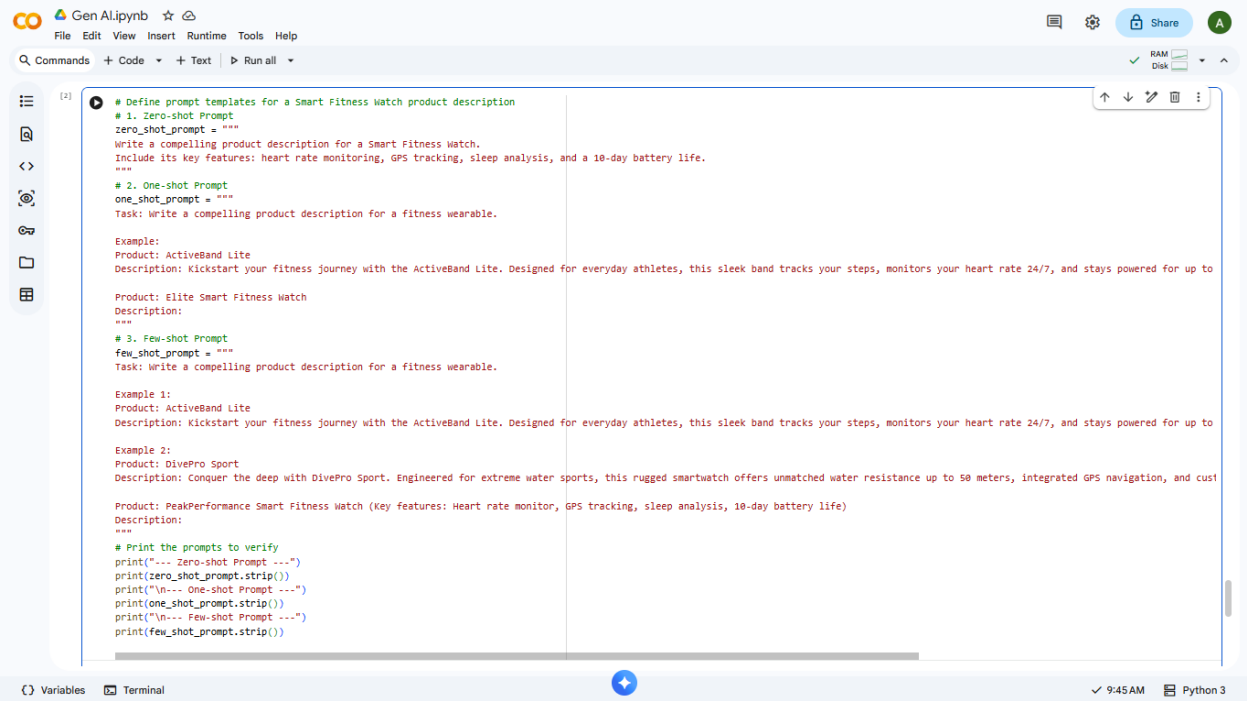
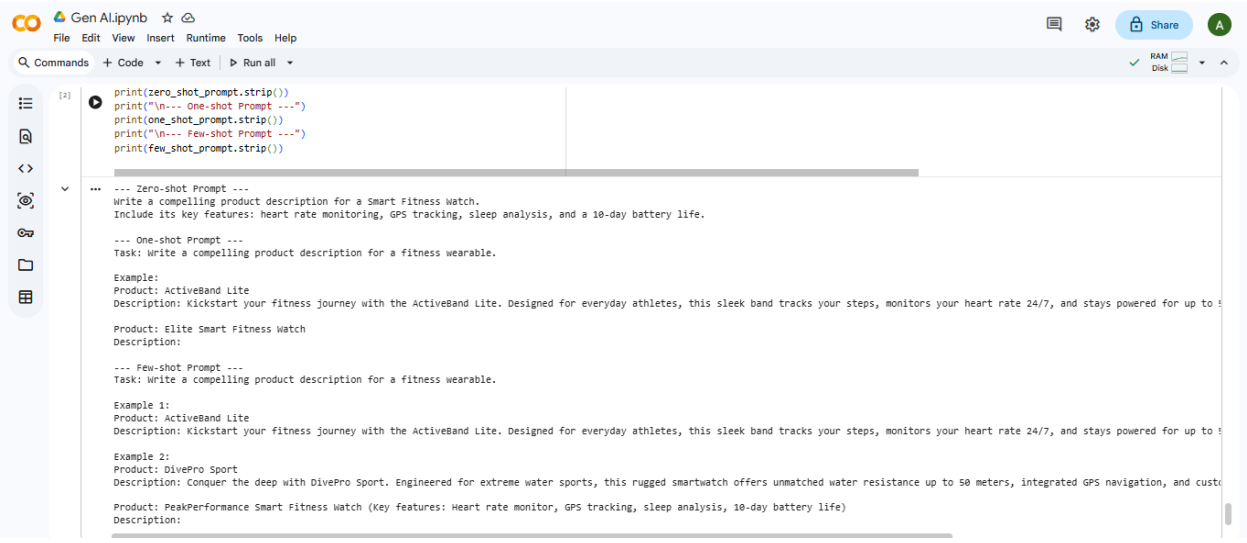
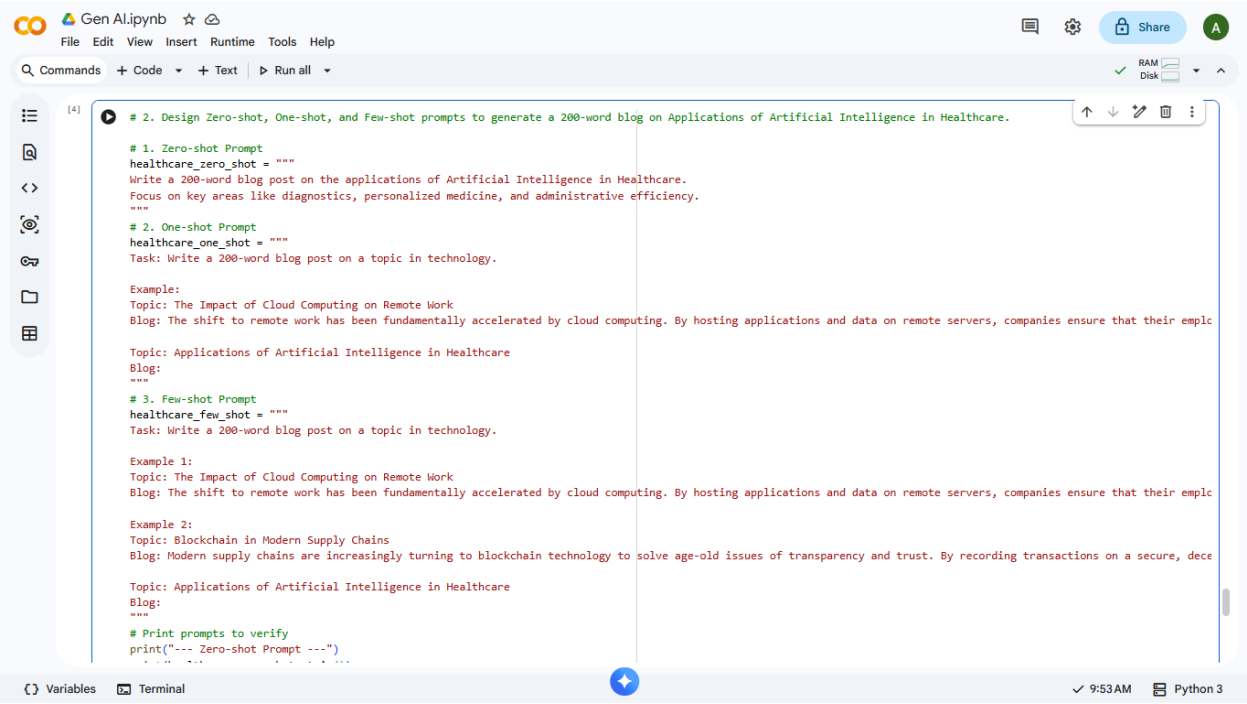
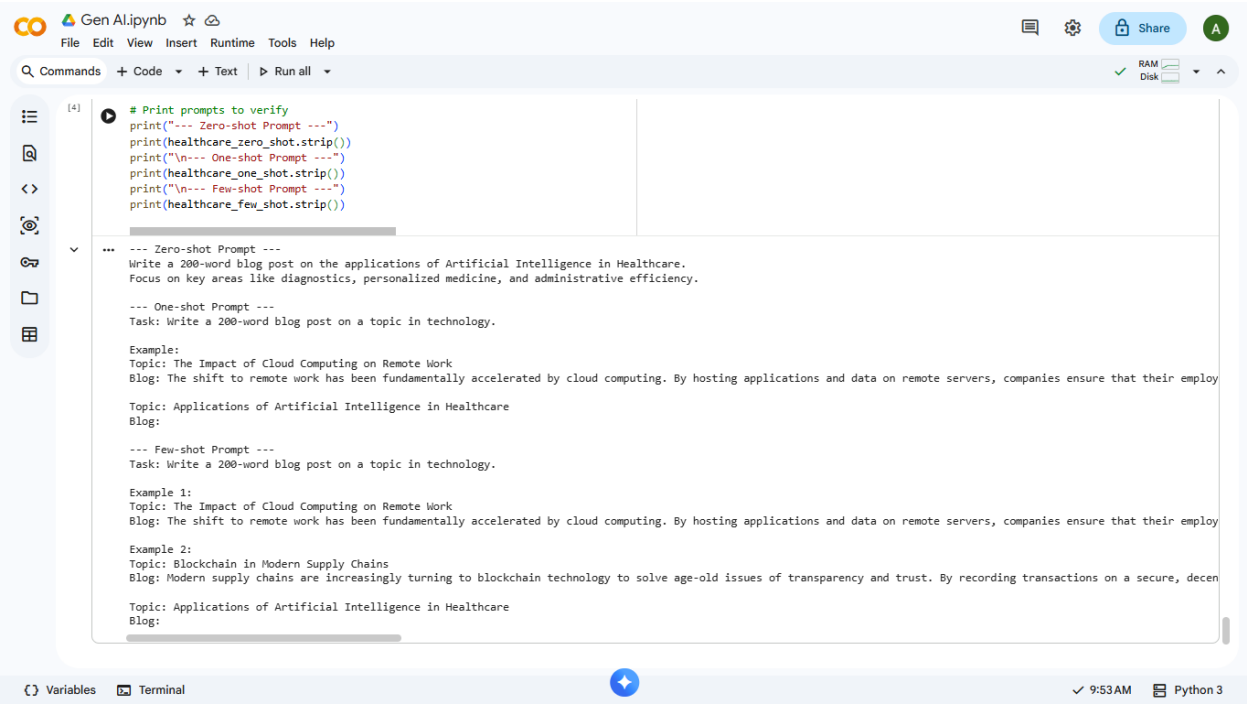
1. Design Zero-shot, One-shot, and Few-shot prompts to generate a product description for a Smart Fitness Watch.



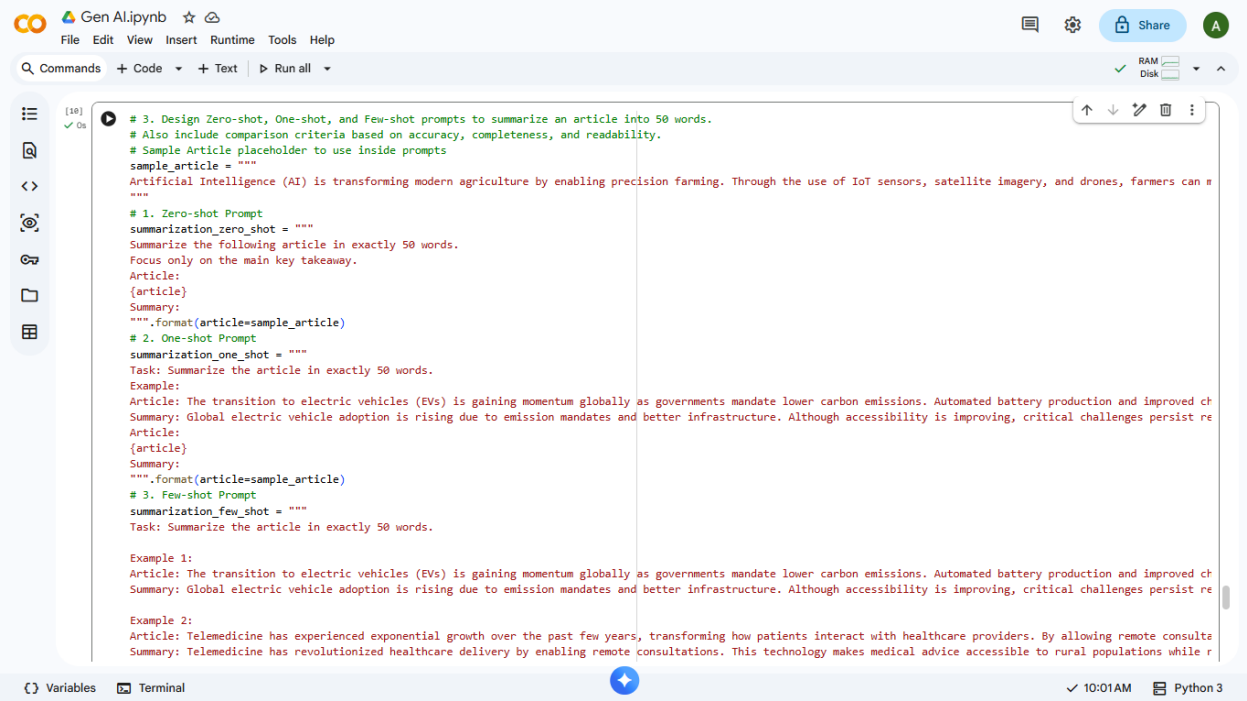



2. Design Zero-shot, One-shot, and Few-shot prompts to generate a 200-word blog on Applications of Artificial Intelligence in Healthcare.



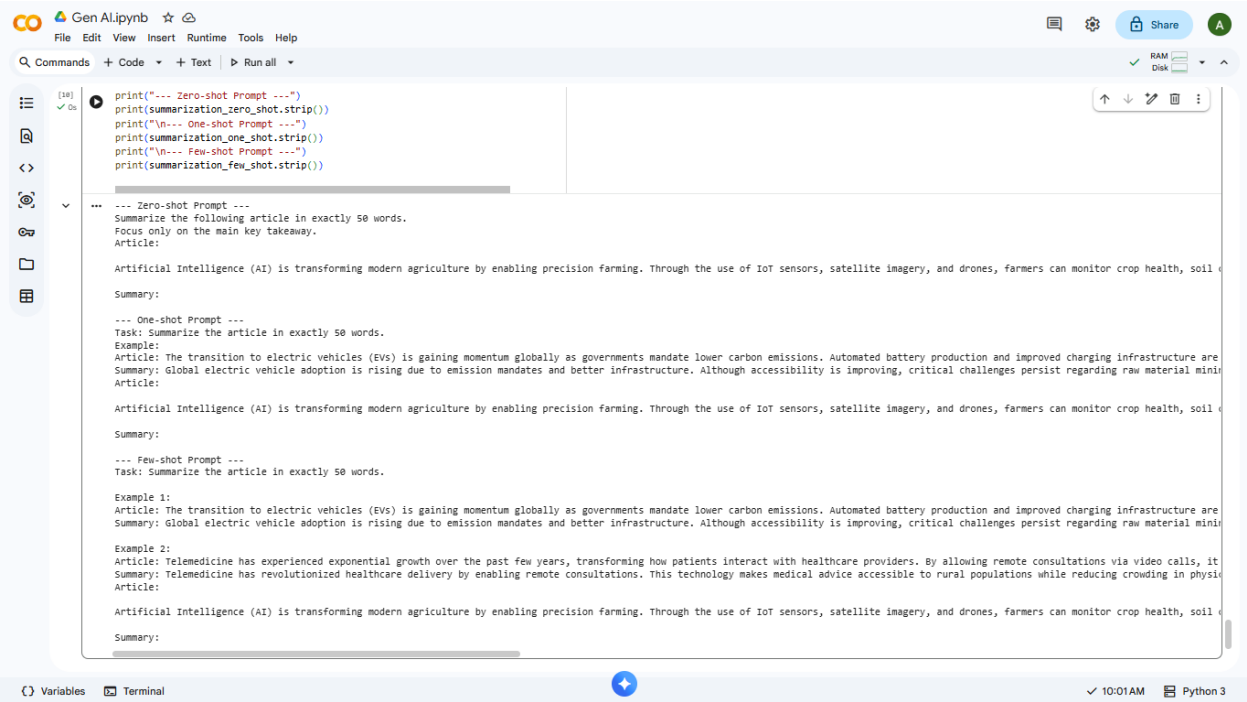
3. Design Zero-shot, One-shot, and Few-shot prompts to summarize the following article into 50 words. Compare the generated summaries based on accuracy, completeness, and readability.



```
# 3. Design Zero-shot, One-shot, and Few-shot prompts to summarize an article into 50 words.
# Also include comparison criteria based on accuracy, completeness, and readability.
# Sample Article placeholder to use inside prompts
sample_article = """
Artificial Intelligence (AI) is transforming modern agriculture by enabling precision farming. Through the use of IoT sensors, satellite imagery, and drones, farmers can m
"""
# 1. Zero-shot Prompt
summarization_zero_shot = """
Summarize the following article in exactly 50 words.
Focus only on the main key takeaway.
Article:
{article}
Summary:
""".format(article=sample_article)
# 2. One-shot Prompt
summarization_one_shot = """
Task: Summarize the article in exactly 50 words.
Example:
Article: The transition to electric vehicles (EVs) is gaining momentum globally as governments mandate lower carbon emissions. Automated battery production and improved ch
Summary: Global electric vehicle adoption is rising due to emission mandates and better infrastructure. Although accessibility is improving, critical challenges persist re
Article:
{article}
Summary:
""".format(article=sample_article)
# 3. Few-shot Prompt
summarization_few_shot = """
Task: Summarize the article in exactly 50 words.

Example 1:
Article: The transition to electric vehicles (EVs) is gaining momentum globally as governments mandate lower carbon emissions. Automated battery production and improved ch
Summary: Global electric vehicle adoption is rising due to emission mandates and better infrastructure. Although accessibility is improving, critical challenges persist re

Example 2:
Article: Telemedicine has experienced exponential growth over the past few years, transforming how patients interact with healthcare providers. By allowing remote consulta
Summary: Telemedicine has revolutionized healthcare delivery by enabling remote consultations. This technology makes medical advice accessible to rural populations while r
```



```
print("--- Zero-shot Prompt ---")
print(summarization_zero_shot.strip())
print("\n--- One-shot Prompt ---")
print(summarization_one_shot.strip())
print("\n--- Few-shot Prompt ---")
print(summarization_few_shot.strip())
```

```
--- Zero-shot Prompt ---
Summarize the following article in exactly 50 words.
Focus only on the main key takeaway.
Article:

Artificial Intelligence (AI) is transforming modern agriculture by enabling precision farming. Through the use of IoT sensors, satellite imagery, and drones, farmers can monitor crop health, soil

Summary:

--- One-shot Prompt ---
Task: Summarize the article in exactly 50 words.
Example:
Article: The transition to electric vehicles (EVs) is gaining momentum globally as governments mandate lower carbon emissions. Automated battery production and improved charging infrastructure are
Summary: Global electric vehicle adoption is rising due to emission mandates and better infrastructure. Although accessibility is improving, critical challenges persist regarding raw material mini
Article:

Artificial Intelligence (AI) is transforming modern agriculture by enabling precision farming. Through the use of IoT sensors, satellite imagery, and drones, farmers can monitor crop health, soil

Summary:

--- Few-shot Prompt ---
Task: Summarize the article in exactly 50 words.

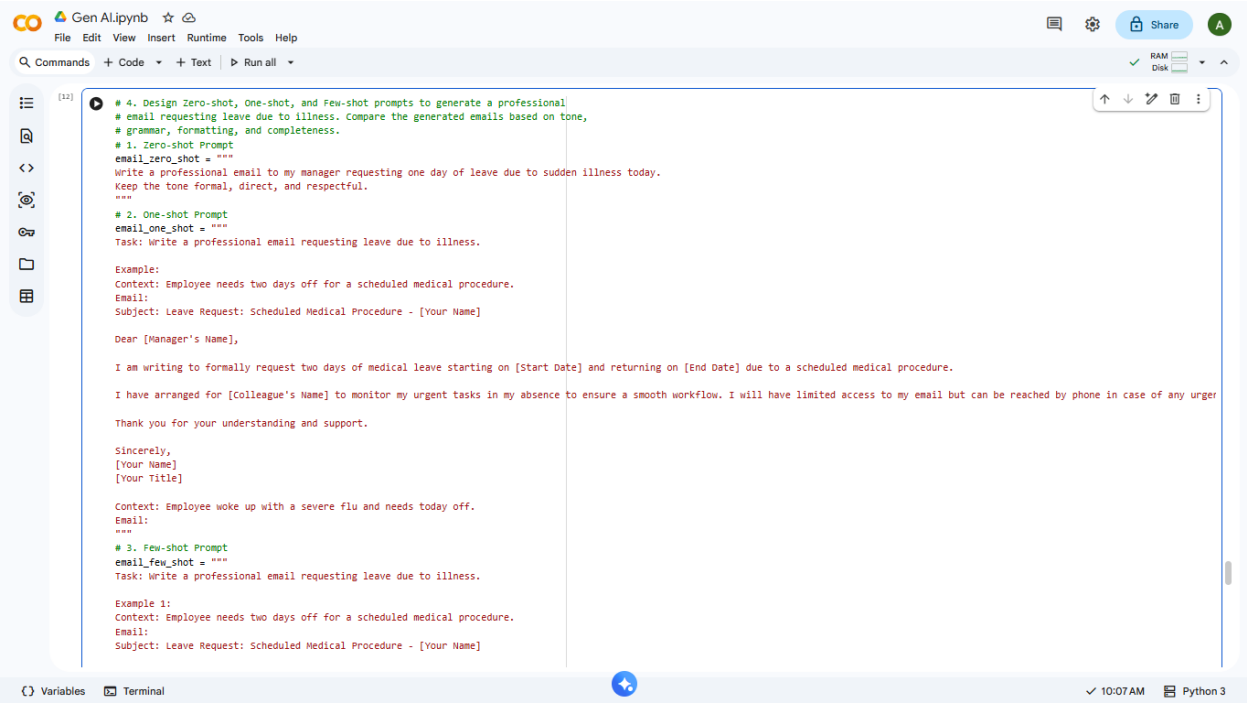
Example 1:
Article: The transition to electric vehicles (EVs) is gaining momentum globally as governments mandate lower carbon emissions. Automated battery production and improved charging infrastructure are
Summary: Global electric vehicle adoption is rising due to emission mandates and better infrastructure. Although accessibility is improving, critical challenges persist regarding raw material mini

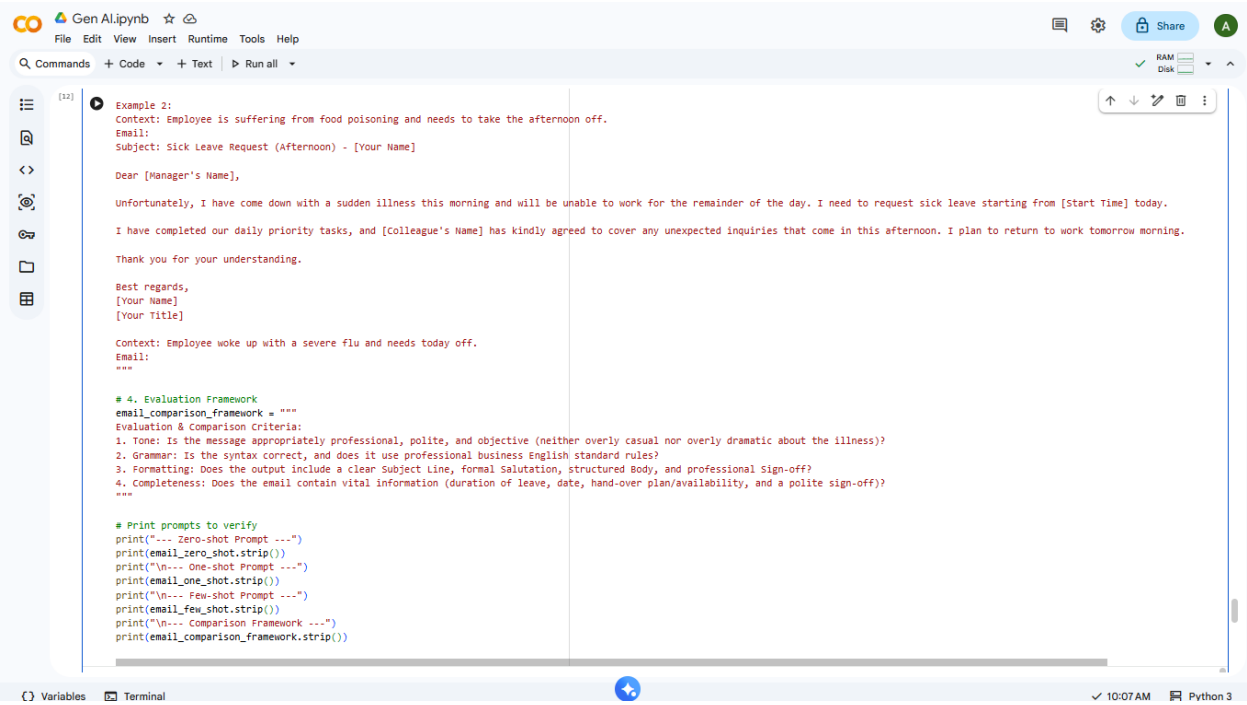
Example 2:
Article: Telemedicine has experienced exponential growth over the past few years, transforming how patients interact with healthcare providers. By allowing remote consultations via video calls, it
Summary: Telemedicine has revolutionized healthcare delivery by enabling remote consultations. This technology makes medical advice accessible to rural populations while reducing crowding in physi
Article:

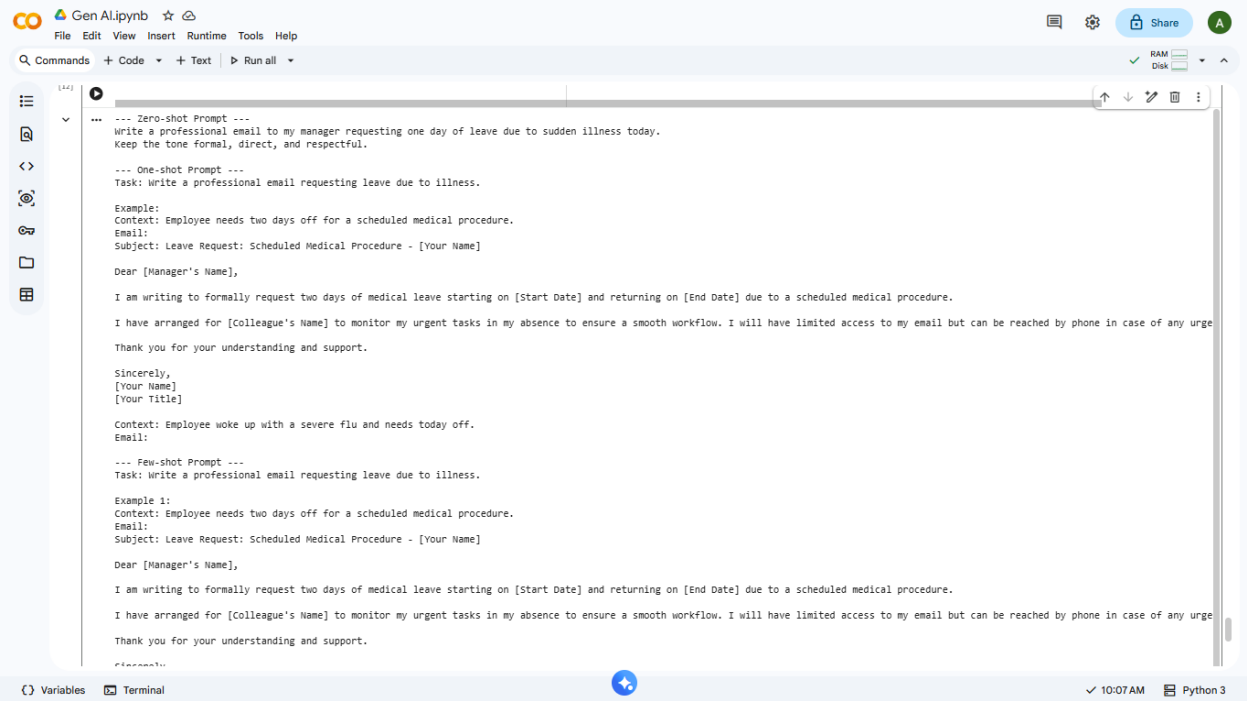
Artificial Intelligence (AI) is transforming modern agriculture by enabling precision farming. Through the use of IoT sensors, satellite imagery, and drones, farmers can monitor crop health, soil

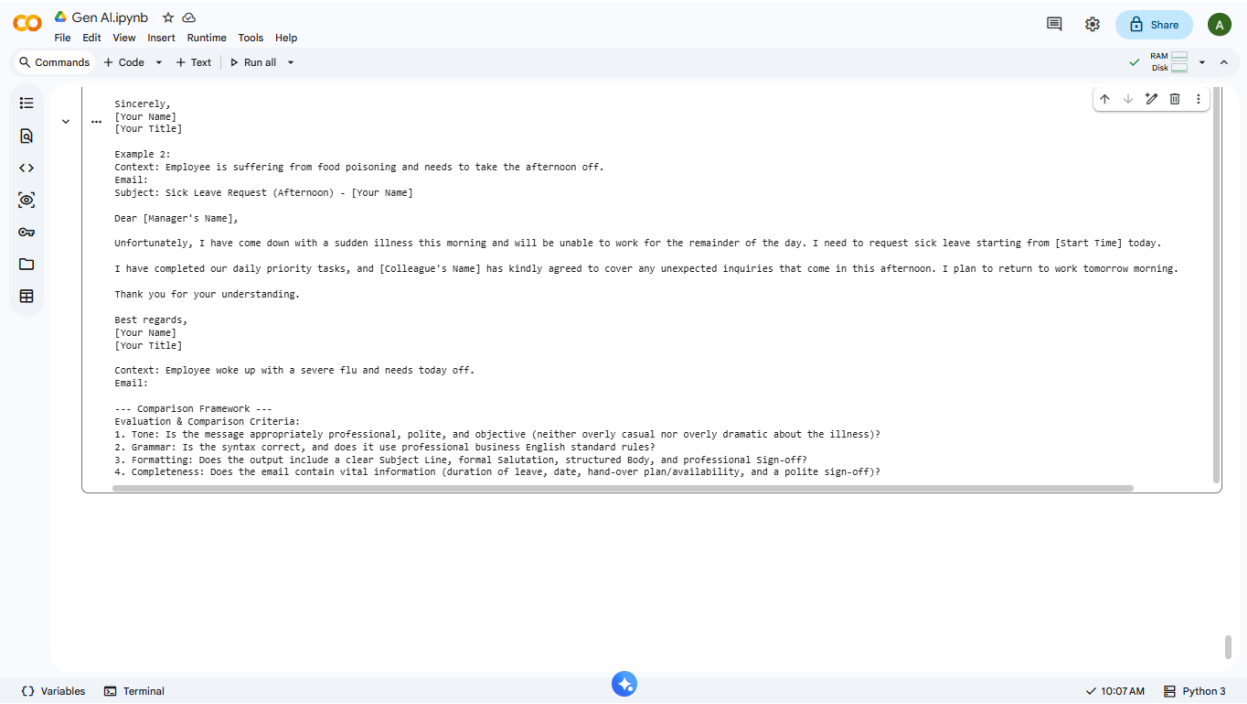
Summary:
```

4. Design Zero-shot, One-shot, and Few-shot prompts to generate a professional email requesting leave due to illness. Compare the generated emails based on tone, grammar, formatting, and completeness.

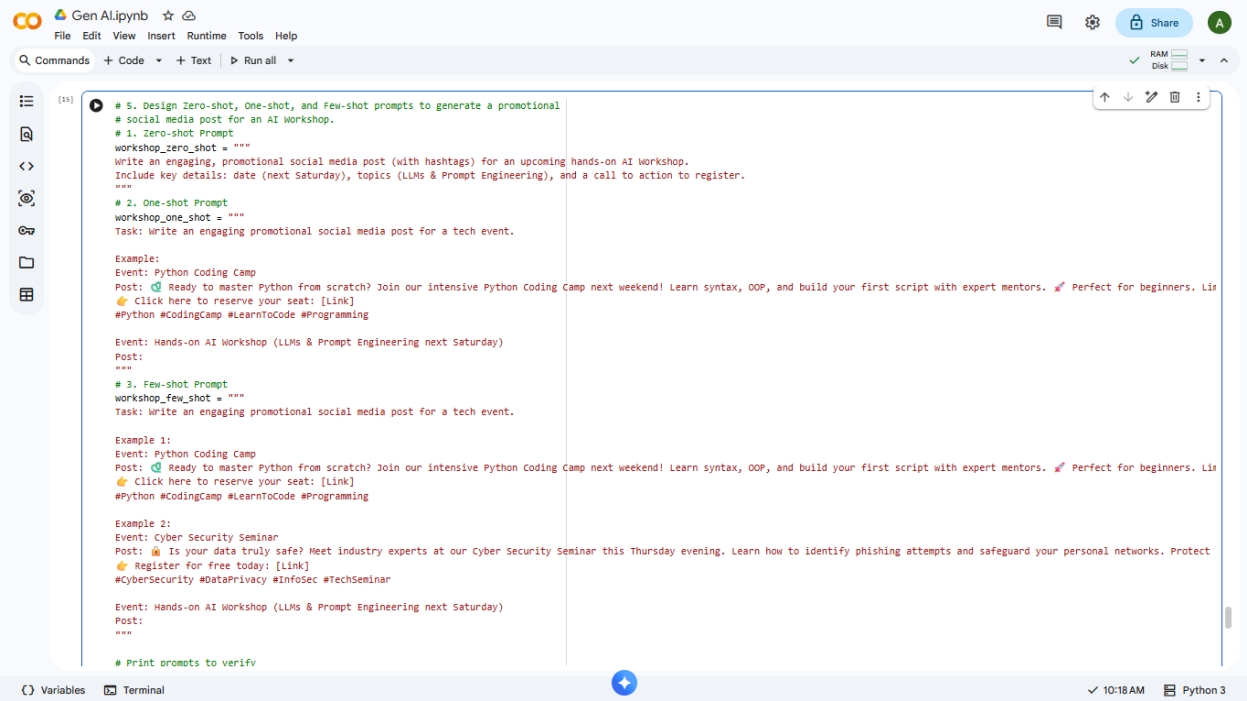


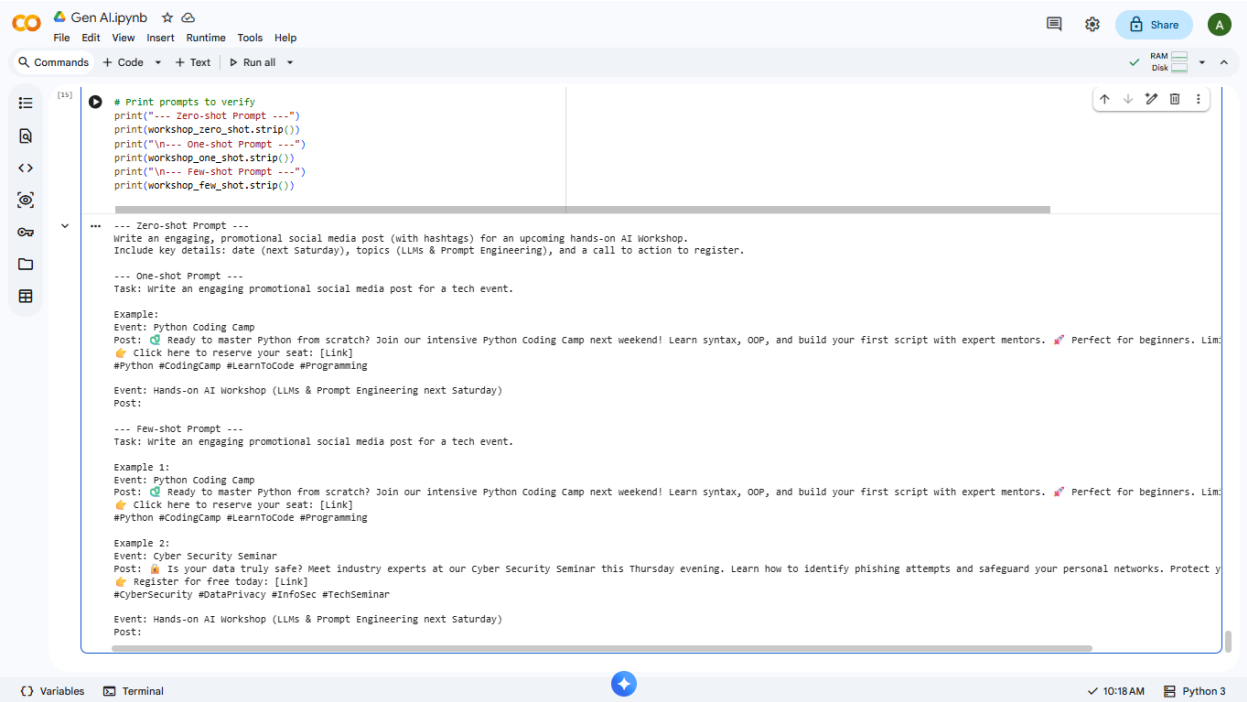




Gen AI.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
--- Zero-shot Prompt ---
Write a professional email to my manager requesting one day of leave due to sudden illness today.
Keep the tone formal, direct, and respectful.
--- One-shot Prompt ---
Task: Write a professional email requesting leave due to illness.
Example:
Context: Employee needs two days off for a scheduled medical procedure.
Email:
Subject: Leave Request: Scheduled Medical Procedure - [Your Name]
Dear [Manager's Name],
I am writing to formally request two days of medical leave starting on [Start Date] and returning on [End Date] due to a scheduled medical procedure.
I have arranged for [Colleague's Name] to monitor my urgent tasks in my absence to ensure a smooth workflow. I will have limited access to my email but can be reached by phone in case of any urge
Thank you for your understanding and support.
Sincerely,
[Your Name]
[Your Title]
Context: Employee woke up with a severe flu and needs today off.
Email:
--- Few-shot Prompt ---
Task: Write a professional email requesting leave due to illness.
Example 1:
Context: Employee needs two days off for a scheduled medical procedure.
Email:
Subject: Leave Request: Scheduled Medical Procedure - [Your Name]
Dear [Manager's Name],
I am writing to formally request two days of medical leave starting on [Start Date] and returning on [End Date] due to a scheduled medical procedure.
I have arranged for [Colleague's Name] to monitor my urgent tasks in my absence to ensure a smooth workflow. I will have limited access to my email but can be reached by phone in case of any urge
Thank you for your understanding and support.
Variables Terminal
10:07 AM Python 3


Gen AI.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
Sincerely,
[Your Name]
[Your Title]
Example 2:
Context: Employee is suffering from food poisoning and needs to take the afternoon off.
Email:
Subject: Sick Leave Request (Afternoon) - [Your Name]
Dear [Manager's Name],
Unfortunately, I have come down with a sudden illness this morning and will be unable to work for the remainder of the day. I need to request sick leave starting from [Start Time] today.
I have completed our daily priority tasks, and [Colleague's Name] has kindly agreed to cover any unexpected inquiries that come in this afternoon. I plan to return to work tomorrow morning.
Thank you for your understanding.
Best regards,
[Your Name]
[Your Title]
Context: Employee woke up with a severe flu and needs today off.
Email:
--- Comparison Framework ---
Evaluation & Comparison Criteria:
1. Tone: Is the message appropriately professional, polite, and objective (neither overly casual nor overly dramatic about the illness)?
2. Grammar: Is the syntax correct, and does it use professional business English standard rules?
3. Formatting: Does the output include a clear Subject Line, formal Salutation, structured Body, and professional Sign-off?
4. Completeness: Does the email contain vital information (duration of leave, date, hand-over plan/availability, and a polite sign-off)?
Variables Terminal
10:07 AM Python 3

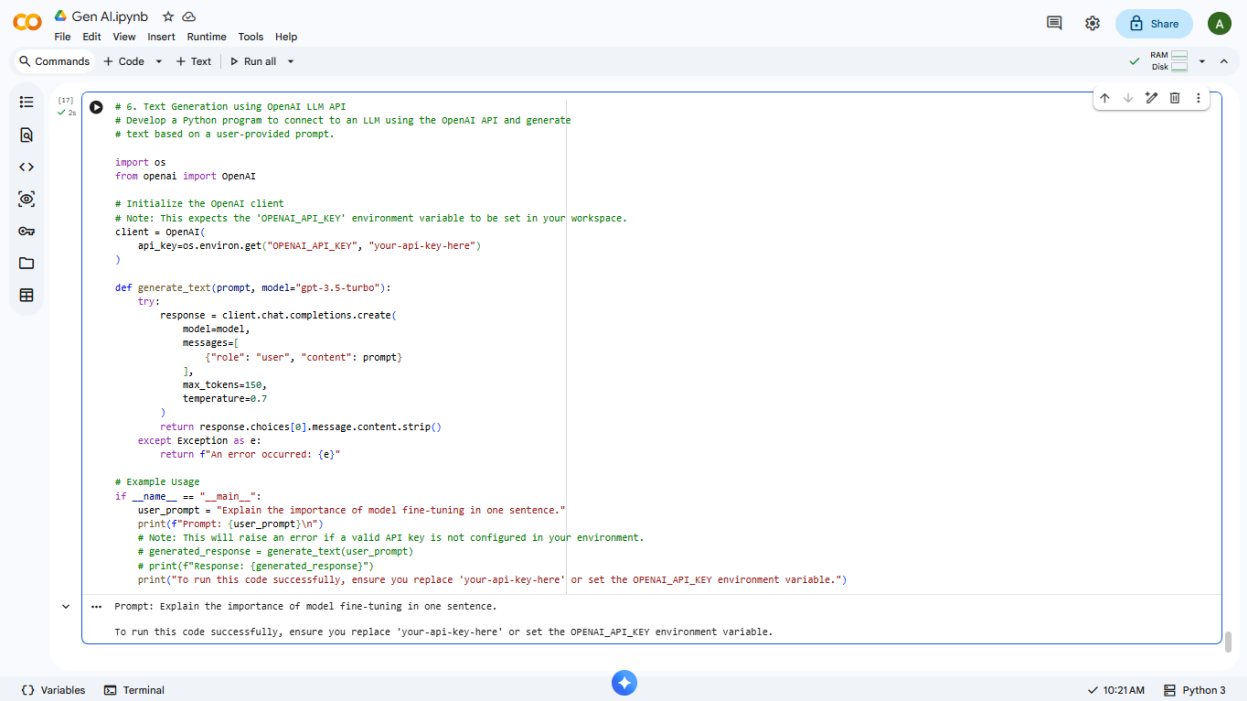
5. Design Zero-shot, One-shot, and Few-shot prompts to generate a promotional social media post for an AI Workshop.



## 6. Text Generation using OpenAI LLM API

Develop a Python program to connect to an LLM using the OpenAI API and generate text based on a user-provided prompt.



```python
# 6. Text Generation using OpenAI LLM API
# Develop a Python program to connect to an LLM using the OpenAI API and generate
# text based on a user-provided prompt.

import os
from openai import OpenAI

# Initialize the OpenAI client
# Note: This expects the 'OPENAI_API_KEY' environment variable to be set in your workspace.
client = OpenAI(
    api_key=os.environ.get("OPENAI_API_KEY", "your-api-key-here")
)

def generate_text(prompt, model="gpt-3.5-turbo"):
    try:
        response = client.chat.completions.create(
            model=model,
            messages=[
                {"role": "user", "content": prompt}
            ],
            max_tokens=150,
            temperature=0.7
        )
        return response.choices[0].message.content.strip()
    except Exception as e:
        return f"An error occurred: {e}"

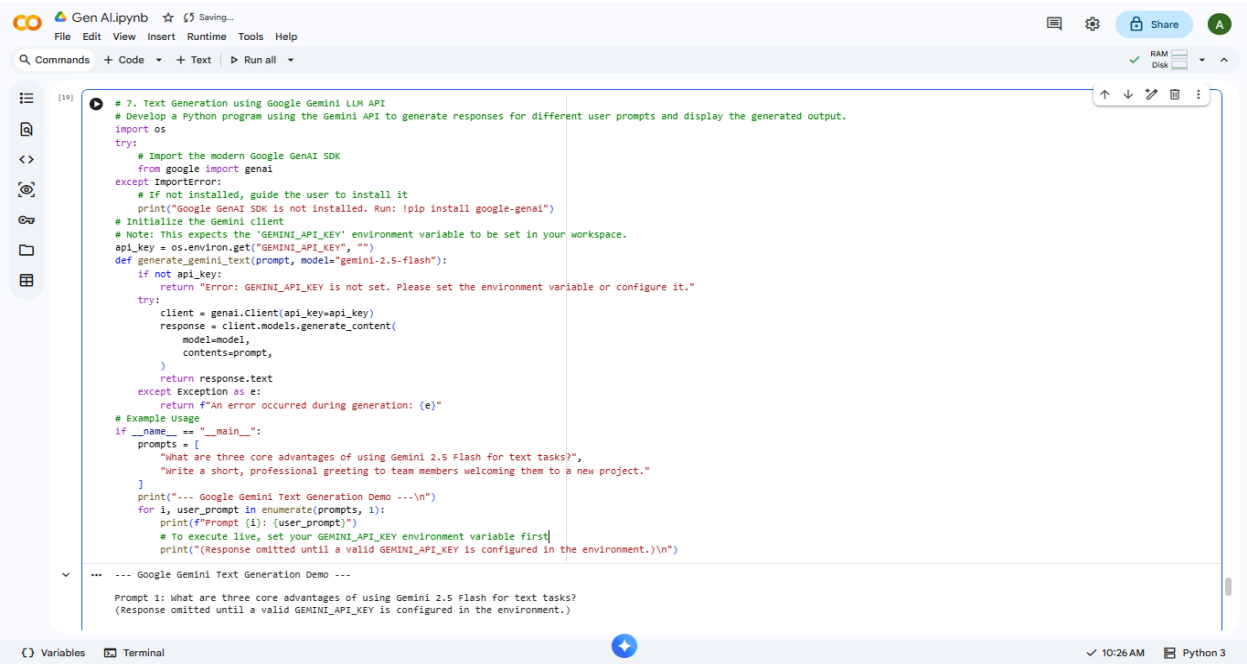
# Example Usage
if __name__ == "__main__":
    user_prompt = "Explain the importance of model fine-tuning in one sentence."
    print(f"Prompt: {user_prompt}\n")
    # Note: This will raise an error if a valid API key is not configured in your environment.
    # generated_response = generate_text(user_prompt)
    # print(f"Response: {generated_response}")
    print("To run this code successfully, ensure you replace 'your-api-key-here' or set the OPENAI_API_KEY environment variable.")
```

```
Prompt: Explain the importance of model fine-tuning in one sentence.

To run this code successfully, ensure you replace 'your-api-key-here' or set the OPENAI_API_KEY environment variable.
```

## 7. Text Generation using Google Gemini LLM API

Develop a Python program using the Gemini API to generate responses for different user prompts and display the generated output.



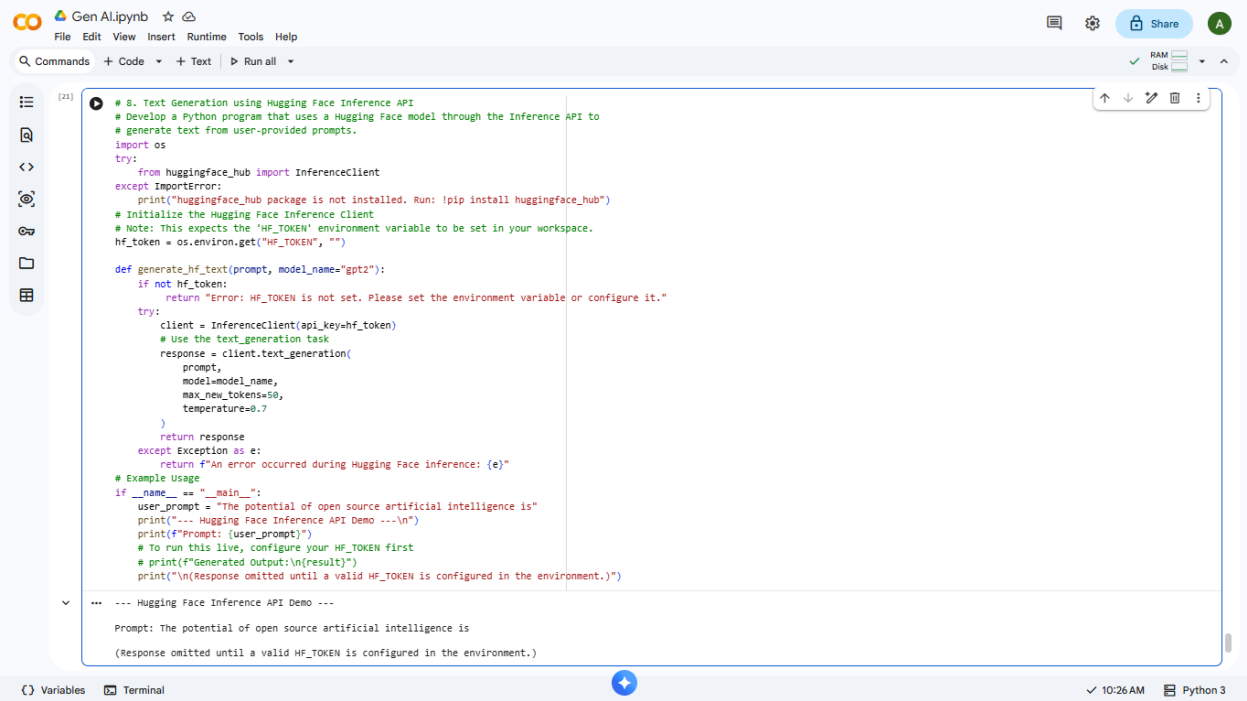
```python
# 7. Text Generation using Google Gemini LLM API
# Develop a Python program using the Gemini API to generate responses for different user prompts and display the generated output.
import os
try:
    # Import the modern Google GenAI SDK
    from google import genai
except ImportError:
    # If not installed, guide the user to install it
    print("Google GenAI SDK is not installed. Run: !pip install google-genai")
# Initialize the Gemini client
# Note: This expects the 'GEMINI_API_KEY' environment variable to be set in your workspace.
api_key = os.environ.get("GEMINI_API_KEY", "")
def generate_gemini_text(prompt, model="gemini-2.5-flash"):
    if not api_key:
        return "Error: GEMINI_API_KEY is not set. Please set the environment variable or configure it."
    try:
        client = genai.Client(api_key=api_key)
        response = client.models.generate_content(
            model=model,
            contents=prompt,
        )
        return response.text
    except Exception as e:
        return f"An error occurred during generation: {e}"
# Example Usage
if __name__ == "__main__":
    prompts = [
        "What are three core advantages of using Gemini 2.5 Flash for text tasks?",
        "Write a short, professional greeting to team members welcoming them to a new project."
    ]
    print("--- Google Gemini Text Generation Demo ---\n")
    for i, user_prompt in enumerate(prompts, 1):
        print(f"Prompt {i}: {user_prompt}")
        # To execute live, set your GEMINI_API_KEY environment variable first
        print("(Response omitted until a valid GEMINI_API_KEY is configured in the environment.)\n")
```

```
--- Google Gemini Text Generation Demo ---

Prompt 1: What are three core advantages of using Gemini 2.5 Flash for text tasks?
(Response omitted until a valid GEMINI_API_KEY is configured in the environment.)
```

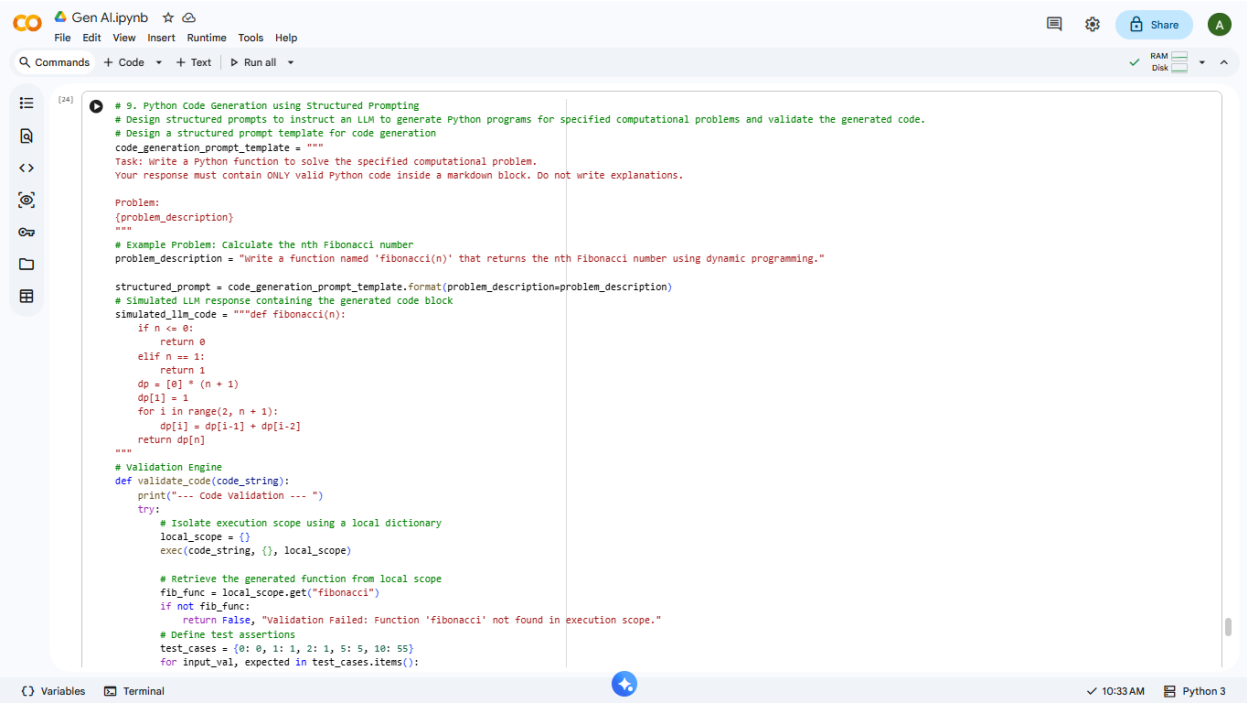
8. Text Generation using Hugging Face Inference API

Develop a Python program that uses a Hugging Face model through the Inference API to generate text from user-provided prompts.



## 9. Python Code Generation using Structured Prompting

Design structured prompts to instruct an LLM to generate Python programs for specified computational problems and validate the generated code.

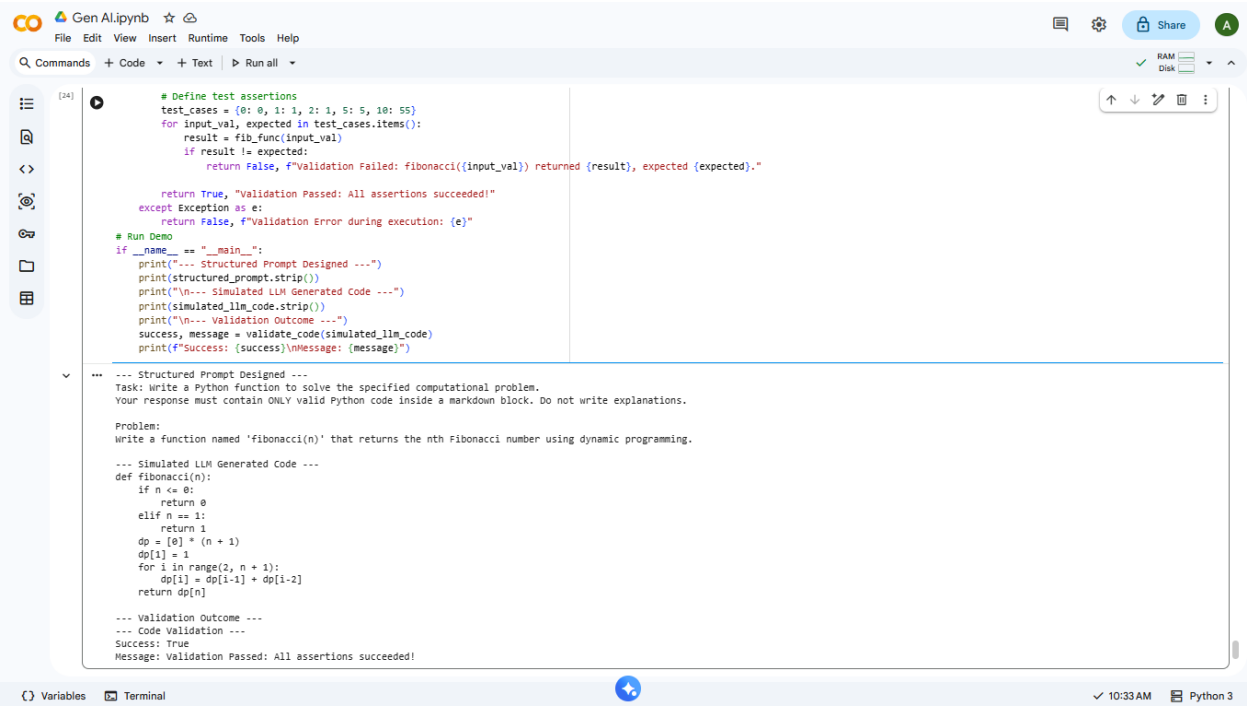


```
# 9. Python Code Generation using Structured Prompting
# Design structured prompts to instruct an LLM to generate Python programs for specified computational problems and validate the generated code.
# Design a structured prompt template for code generation
code_generation_prompt_template = """
Task: Write a Python function to solve the specified computational problem.
Your response must contain ONLY valid Python code inside a markdown block. Do not write explanations.

Problem:
{problem_description}
"""
# Example Problem: Calculate the nth Fibonacci number
problem_description = "Write a function named 'fibonacci(n)' that returns the nth Fibonacci number using dynamic programming."

structured_prompt = code_generation_prompt_template.format(problem_description=problem_description)
# Simulated LLM response containing the generated code block
simulated_llm_code = """def fibonacci(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
"""
# Validation Engine
def validate_code(code_string):
    print("--- Code Validation --- ")
    try:
        # Isolate execution scope using a local dictionary
        local_scope = {}
        exec(code_string, {}, local_scope)

        # Retrieve the generated function from local scope
        fib_func = local_scope.get("fibonacci")
        if not fib_func:
            return False, "Validation Failed: Function 'fibonacci' not found in execution scope."
        # Define test assertions
        test_cases = {0: 0, 1: 1, 2: 1, 5: 5, 10: 55}
        for input_val, expected in test_cases.items():
```



```
        # Define test assertions
        test_cases = {0: 0, 1: 1, 2: 1, 5: 5, 10: 55}
        for input_val, expected in test_cases.items():
            result = fib_func(input_val)
            if result != expected:
                return False, f"Validation Failed: fibonacci({input_val}) returned {result}, expected {expected}."

        return True, "Validation Passed: All assertions succeeded!"
    except Exception as e:
        return False, f"Validation Error during execution: {e}"
# Run Demo
if __name__ == "__main__":
    print("--- Structured Prompt Designed ---")
    print(structured_prompt.strip())
    print("\n--- Simulated LLM Generated Code ---")
    print(simulated_llm_code.strip())
    print("\n--- Validation Outcome ---")
    success, message = validate_code(simulated_llm_code)
    print(f"Success: {success}\nMessage: {message}")
```

```
--- Structured Prompt Designed ---
Task: Write a Python function to solve the specified computational problem.
Your response must contain ONLY valid Python code inside a markdown block. Do not write explanations.

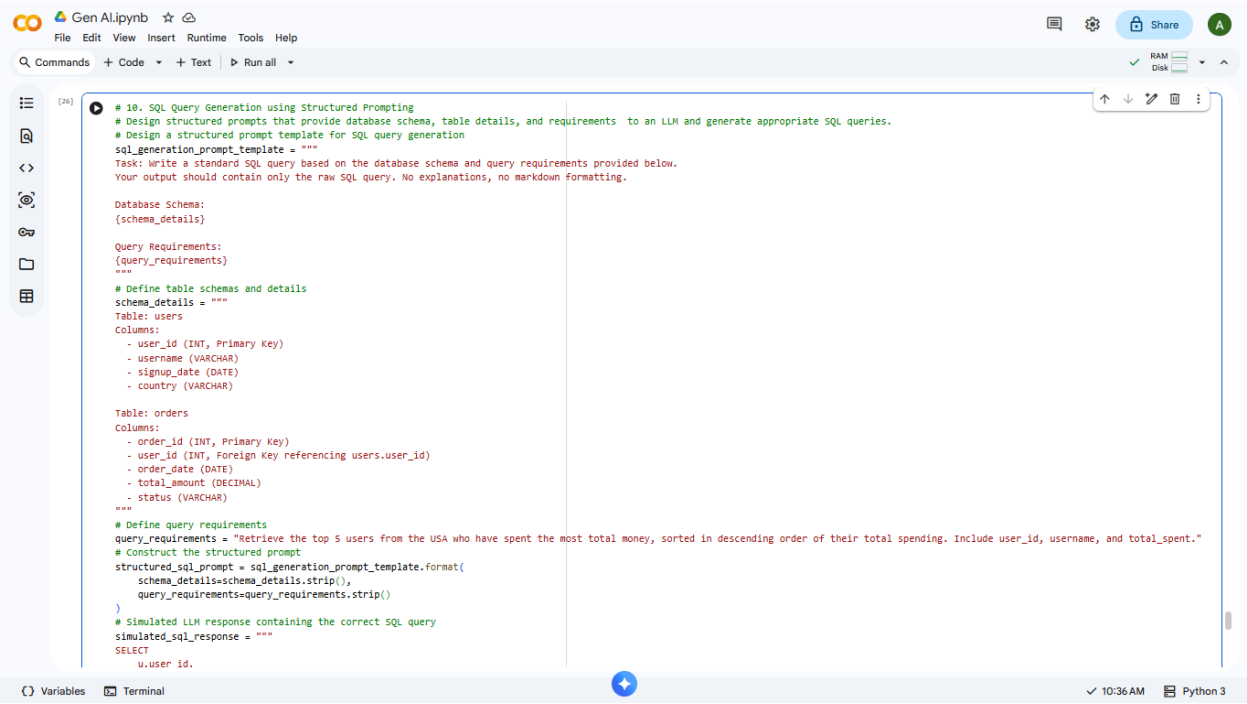
Problem:
Write a function named 'fibonacci(n)' that returns the nth Fibonacci number using dynamic programming.

--- Simulated LLM Generated Code ---
def fibonacci(n):
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

--- Validation Outcome ---
--- Code Validation ---
Success: True
Message: Validation Passed: All assertions succeeded!
```

## 10. SQL Query Generation using Structured Prompting

Design structured prompts that provide database schema, table details, and requirements to an LLM and generate appropriate SQL queries.

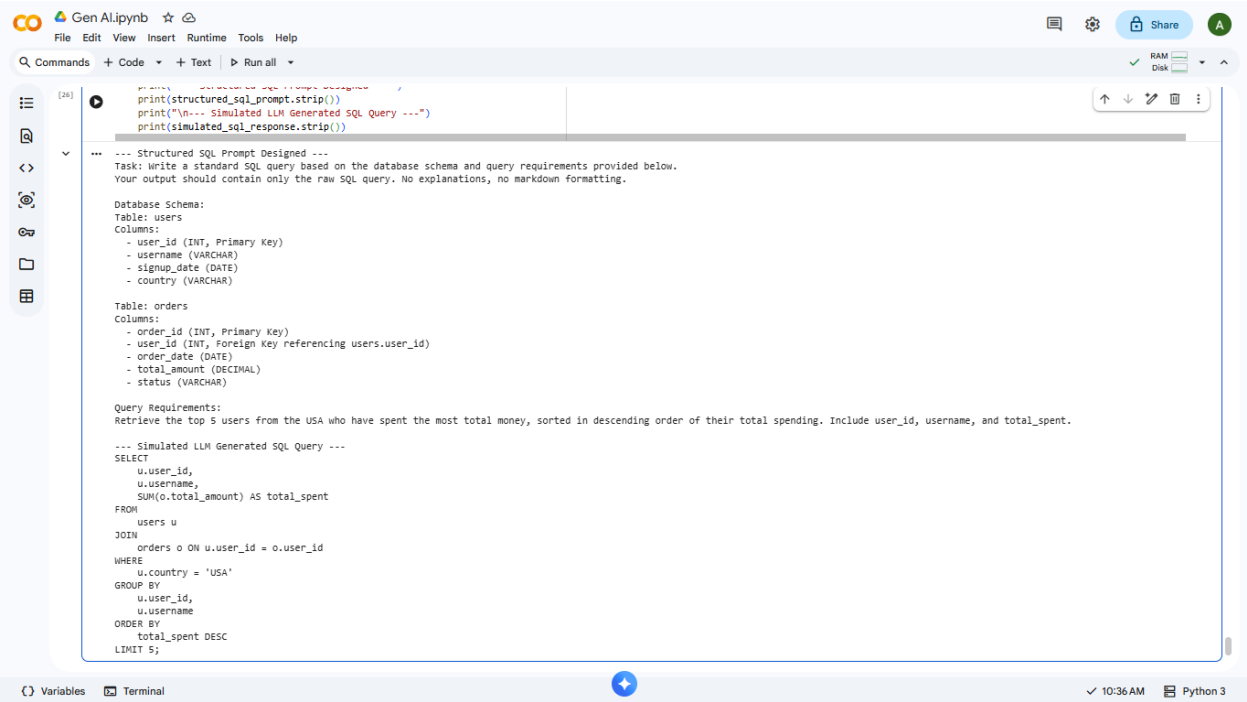


```
# 10. SQL Query Generation using Structured Prompting
# Design structured prompts that provide database schema, table details, and requirements  to an LLM and generate appropriate SQL queries.
# Design a structured prompt template for SQL query generation
sql_generation_prompt_template = """
Task: Write a standard SQL query based on the database schema and query requirements provided below.
Your output should contain only the raw SQL query. No explanations, no markdown formatting.

Database Schema:
{schema_details}

Query Requirements:
{query_requirements}
"""
# Define table schemas and details
schema_details = """
Table: users
Columns:
  - user_id (INT, Primary Key)
  - username (VARCHAR)
  - signup_date (DATE)
  - country (VARCHAR)

Table: orders
Columns:
  - order_id (INT, Primary Key)
  - user_id (INT, Foreign Key referencing users.user_id)
  - order_date (DATE)
  - total_amount (DECIMAL)
  - status (VARCHAR)
"""
# Define query requirements
query_requirements = "Retrieve the top 5 users from the USA who have spent the most total money, sorted in descending order of their total spending. Include user_id, username, and total_spent."
# Construct the structured prompt
structured_sql_prompt = sql_generation_prompt_template.format(
    schema_details=schema_details.strip(),
    query_requirements=query_requirements.strip()
)
# Simulated LLM response containing the correct SQL query
simulated_sql_response = """
SELECT
    u.user_id,
```



```
print(structured_sql_prompt.strip())
print("\n--- Simulated LLM Generated SQL Query ---")
print(simulated_sql_response.strip())
```

```
--- Structured SQL Prompt Designed ---
Task: Write a standard SQL query based on the database schema and query requirements provided below.
Your output should contain only the raw SQL query. No explanations, no markdown formatting.

Database Schema:
Table: users
Columns:
  - user_id (INT, Primary Key)
  - username (VARCHAR)
  - signup_date (DATE)
  - country (VARCHAR)

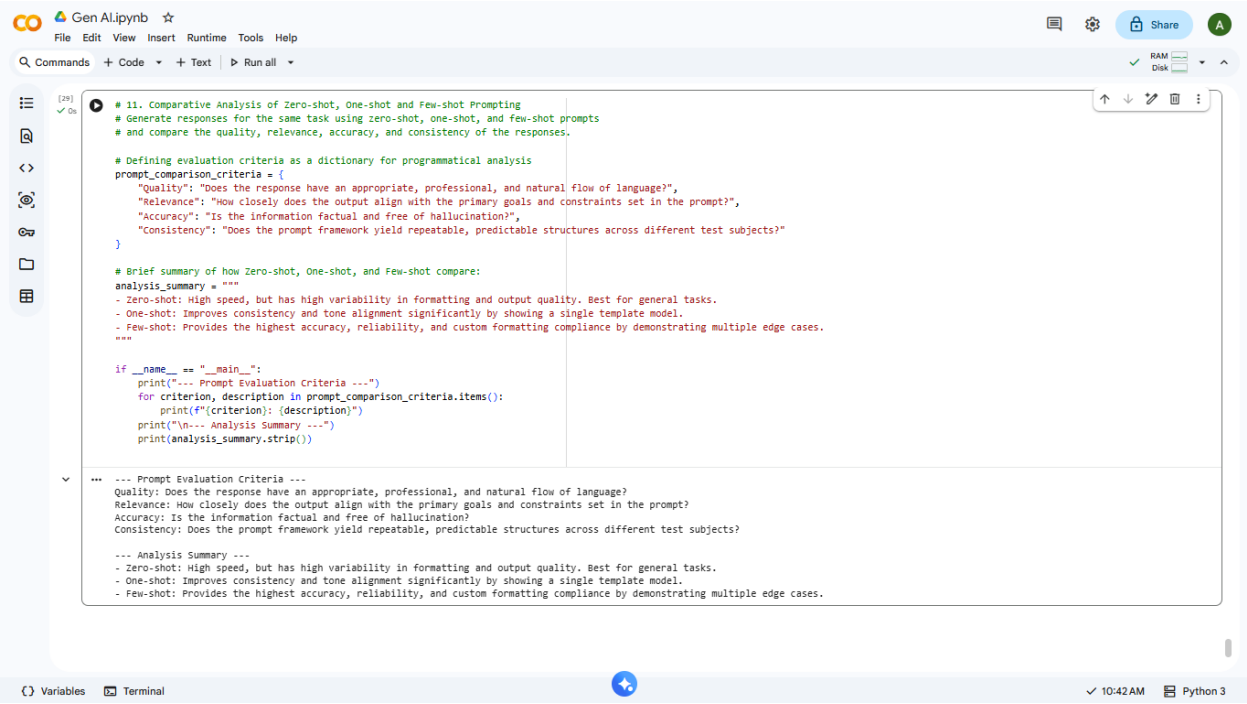
Table: orders
Columns:
  - order_id (INT, Primary Key)
  - user_id (INT, Foreign Key referencing users.user_id)
  - order_date (DATE)
  - total_amount (DECIMAL)
  - status (VARCHAR)

Query Requirements:
Retrieve the top 5 users from the USA who have spent the most total money, sorted in descending order of their total spending. Include user_id, username, and total_spent.

--- Simulated LLM Generated SQL Query ---
SELECT
    u.user_id,
    u.username,
    SUM(o.total_amount) AS total_spent
FROM
    users u
JOIN
    orders o ON u.user_id = o.user_id
WHERE
    u.country = 'USA'
GROUP BY
    u.user_id,
    u.username
ORDER BY
    total_spent DESC
LIMIT 5;
```

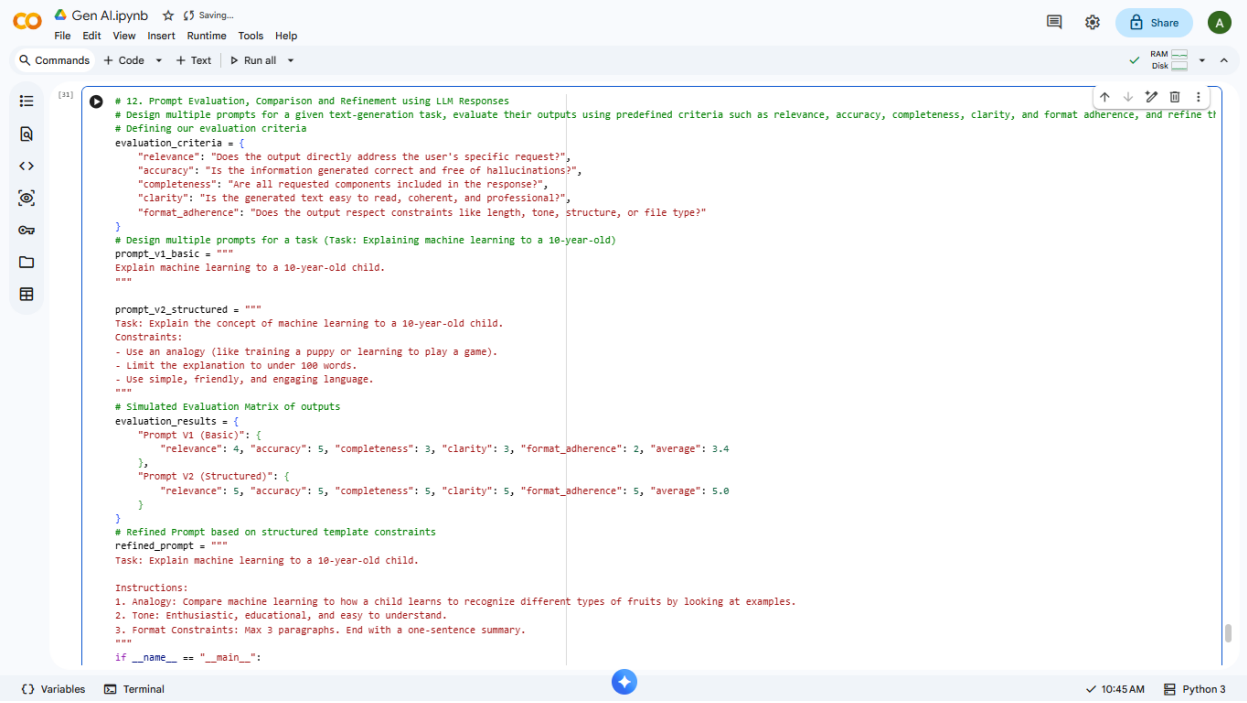
11. Comparative Analysis of Zero-shot, One-shot and Few-shot Prompting

Generate responses for the same task using zero-shot, one-shot, and few-shot prompts and compare the quality, relevance, accuracy, and consistency of the responses.



## 12. Prompt Evaluation, Comparison and Refinement using LLM Responses

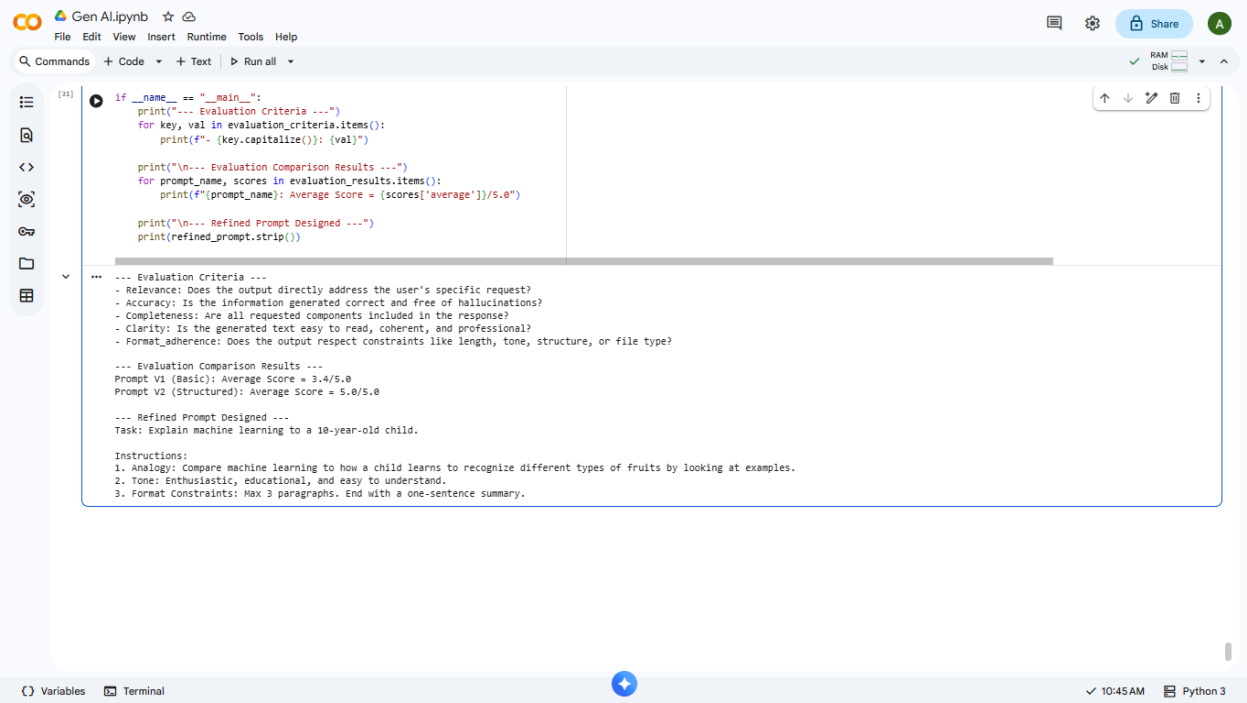
Design multiple prompts for a given text-generation task, evaluate their outputs using predefined criteria such as relevance, accuracy, completeness, clarity, and format adherence, and refine the best-performing prompt.



```
# 12. Prompt Evaluation, Comparison and Refinement using LLM Responses
# Design multiple prompts for a given text-generation task, evaluate their outputs using predefined criteria such as relevance, accuracy, completeness, clarity, and format adherence, and refine t
# Defining our evaluation criteria
evaluation_criteria = {
    "relevance": "Does the output directly address the user's specific request?",
    "accuracy": "Is the information generated correct and free of hallucinations?",
    "completeness": "Are all requested components included in the response?",
    "clarity": "Is the generated text easy to read, coherent, and professional?",
    "format_adherence": "Does the output respect constraints like length, tone, structure, or file type?"
}
# Design multiple prompts for a task (Task: Explaining machine learning to a 10-year-old)
prompt_v1_basic = """
Explain machine learning to a 10-year-old child.
"""

prompt_v2_structured = """
Task: Explain the concept of machine learning to a 10-year-old child.
Constraints:
- Use an analogy (like training a puppy or learning to play a game).
- Limit the explanation to under 100 words.
- Use simple, friendly, and engaging language.
"""
# Simulated Evaluation Matrix of outputs
evaluation_results = {
    "Prompt V1 (Basic)": {
        "relevance": 4, "accuracy": 5, "completeness": 3, "clarity": 3, "format_adherence": 2, "average": 3.4
    },
    "Prompt V2 (Structured)": {
        "relevance": 5, "accuracy": 5, "completeness": 5, "clarity": 5, "format_adherence": 5, "average": 5.0
    }
}
# Refined Prompt based on structured template constraints
refined_prompt = """
Task: Explain machine learning to a 10-year-old child.

Instructions:
1. Analogy: Compare machine learning to how a child learns to recognize different types of fruits by looking at examples.
2. Tone: Enthusiastic, educational, and easy to understand.
3. Format Constraints: Max 3 paragraphs. End with a one-sentence summary.
"""
if __name__ == "__main__":
```



```
if __name__ == "__main__":
    print("--- Evaluation Criteria ---")
    for key, val in evaluation_criteria.items():
        print(f"- {key.capitalize()}: {val}")

    print("\n--- Evaluation Comparison Results ---")
    for prompt_name, scores in evaluation_results.items():
        print(f"{prompt_name}: Average Score = {scores['average']}/5.0")

    print("\n--- Refined Prompt Designed ---")
    print(refined_prompt.strip())
```

```
--- Evaluation Criteria ---
- Relevance: Does the output directly address the user's specific request?
- Accuracy: Is the information generated correct and free of hallucinations?
- Completeness: Are all requested components included in the response?
- Clarity: Is the generated text easy to read, coherent, and professional?
- Format_adherence: Does the output respect constraints like length, tone, structure, or file type?

--- Evaluation Comparison Results ---
Prompt V1 (Basic): Average Score = 3.4/5.0
Prompt V2 (Structured): Average Score = 5.0/5.0

--- Refined Prompt Designed ---
Task: Explain machine learning to a 10-year-old child.

Instructions:
1. Analogy: Compare machine learning to how a child learns to recognize different types of fruits by looking at examples.
2. Tone: Enthusiastic, educational, and easy to understand.
3. Format Constraints: Max 3 paragraphs. End with a one-sentence summary.
```